ASTL API Specification

C API, Python Wrapper, Go Wrapper, and Discovery Model for ASTL 0.0.1

Arm SoC Telemetry Library (ASTL)

2026-04-03

Contents

0.1 Purpose . . . 2
0.2 Version Covered . . . 3
0.3 Architecture Summary . . . 3
0.4 API Layers . . . 4
0.4.1 C API . . . 4
0.4.2 Python Wrapper . . . 4
0.4.3 Go Wrapper . . . 4
0.5 Version API . . . 5
0.6 Core C API Conventions . . . 5
0.6.1 Struct Versioning . . . 5
0.6.2 Caller-Allocated Buffers . . . 5
0.6.3 Opaque handles . . . 6
0.6.4 Status Codes . . . 6
0.6.5 Ownership and Lifetimes . . . 7
0.7 Public Data Model . . . 7
0.7.1 Platform . . . 7
0.7.2 Target . . . 7
0.7.3 Counter . . . 7
0.7.4 Metric . . . 7
0.7.5 Metric Group . . . 8
0.7.6 Sample . . . 8
0.8 Shared Enums and Semantic Types . . . 9
0.8.1 Units . . . 9
0.8.2 Value Types . . . 9
0.8.3 Metric Identifiers . . . 9
0.8.4 Collection Modes . . . 10
0.9 C API Inventory by Area . . . 10
0.9.1 System Information . . . 10
0.9.2 Target Discovery . . . 10
0.9.3 Counter Discovery . . . 10
0.9.4 Metric Discovery . . . 10
0.9.5 Metric State Discovery . . . 10
0.9.6 Metric-Group Discovery . . . 10

0.9.7 Collection Configuration . . . 11
0.9.8 Collection Lifecycle . . . 11
0.9.9 Session Persistence . . . 11
0.9.10 Sample Retrieval . . . 11
0.9.11 Statistics, Histogram, And States . . . 11
0.9.12 Cropping . . . 12
0.10 Typical Native Integration Flow . . . 12
0.10.1 Flow 1: Enumerate The Platform . . . 12
0.10.2 Flow 2: Configure And Collect Metrics . . . 12
0.10.3 Flow 2a: Configure And Read Immediately . . . 12
0.10.4 Flow 3: Save And Reload . . . 12
0.11 Example C Discovery Pattern . . . 12
0.12 Python Wrapper Specification . . . 13
0.12.1 Python Wrapper Positioning . . . 13
0.12.2 Public Modules . . . 13
0.12.3 Current Public Capabilities . . . 13
0.12.4 Installation And Runtime Expectations . . . 14
0.13 Go Wrapper Specification . . . 15
0.13.1 Go Wrapper Positioning . . . 15
0.13.2 Current Exported Surface . . . 15
0.13.3 Build And Link Requirements . . . 16
0.14 Low-Level Telemetry Sources . . . 16
0.14.1 SCMI . . . 16
0.14.2 Libsensors . . . 18
0.15 ASTL Runtime Configuration Resolution . . . 19
0.16 What Dynamic Discovery Means in Practice . . . 20
0.16.1 Live Host Discovery . . . 20
0.16.2 Loaded-Session Mode . . . 20
0.17 Supported Platforms . . . 20
0.17.1 Arm AGI CPU . . . 21
0.17.2 Ampere Altra . . . 22
0.18 Release Readiness Review . . . 23
0.18.1 Blind Spots And Gaps To Address Before First Release . . . 23
0.18.2 Pre-Release Compatibility Code Currently Present . . . 24
0.19 Release Notes . . . 24
0.20 Recommended Consumer Guidance . . . 24
0.20.1 If You Want The Most Stable Integration Surface . . . 24
0.20.2 If You Want The Most Ergonomic Scripting And Analysis Layer . . . 24
0.20.3 If You Want Native Go And Want A Wrapper Close To The C API . . . 24
0.21 Notes and Caveats . . . 25
0.22 Reference Pointers . . . 25

0.1 Purpose

This document specifies the current public API surface of the Arm SoC Telemetry Library (ASTL), including:

- The core C API,

- The current Python wrapper,
- The current Go wrapper,
- The low-level discovery model for SCMI and libsensors-backed telemetry.

The intent is to give integrators and reviewers a practical, versioned reference without requiring them to read the full source tree first.

0.2 Version Covered

This specification covers the current ASTL version:

- `VERSION.md`: `0.0.1`
- Runtime version APIs: `astlVersionString()` and `astlVersion()`

The version is therefore:

- **ASTL 0.0.1**

Primary source files used to build this document:

- `include/astl/astl.h`
- `include/astl/astl_telemetry.h`
- `include/astl/astl_errors.h`
- `include/astl/astl_version.h.in`
- `README.md`
- `python/astl/__init__.py`
- `python/astl/_core.pyx`
- `python/doc/README.md`
- `python/doc/USER_GUIDE.md`
- `Go/README.md`
- `Go/astl/astl.go`

0.3 Architecture Summary

ASTL is a C API with a C++ implementation. It abstracts multiple low-level telemetry data sources and presents them through a single discovery, configuration, lifecycle, and retrieval interface.

Current stack:

1. Low-level telemetry backends
   - SCMI via Linux SCMI telemetry ioctl and legacy sysfs
   - Hwmon telemetry via libsensors
2. ASTL internal discovery and metric-building layers
3. Public C API
4. Language wrappers
   - Python wrapper
   - Go wrapper
5. Tools built on top
   - For example, `atx`

0.4 API Layers

0.4.1 C API

The stable entry point for native consumers is:

```
#include "astl/astl.h"
```

This umbrella header includes:

- `astl_telemetry.h`
- `astl_errors.h`
- `astl_version.h`

Pure C callers can also include `astl_telemetry.h` directly.

0.4.2 Python Wrapper

The Python package lives under `python/astl`.

It layers an ergonomic API on top of the C API.

Current structure:

- `astl._core`: thin Cython bridge to the C API
- `astl`: curated public facade
- `astl.streaming`: sync and async polling helpers
- `astl.session`: context-managed collection lifecycle
- `astl.diagnostics`: environment diagnostics
- `astl.dataframe`: optional pandas integration
- `astl.derived`: delta and rate helpers
- `astl.exceptions`: mapped Python exception hierarchy

The Python package exposes `__version__` and falls back to the repository's `VERSION.md` when distribution metadata is unavailable.

0.4.3 Go Wrapper

The Go package lives under `Go/astl`.

It uses `cgo` to call the public ASTL C API directly and expects:

- The ASTL public headers,
- A built ASTL shared library,
- Compatible linker and runtime loader paths.

The Go wrapper is best understood as a native Go façade over the same C API concepts:

- Version and status helpers,
- Target, counter, metric, and metric-group discovery,
- Collection configuration,
- Lifecycle control,
- Save/load,
- Sample retrieval,
- Statistics, histogram, and state access,

- Crop operations.

0.5 Version API

ASTL exports two version helpers:

- `astlVersionString()`: Return the library semantic version as a string.
- `astlVersion()`: Return `{major, minor, micro}` as `astl_version_t`.

Client applications also compile against version macros from `astl_version.h`, including:

```
#ifndef ASTL_VERSION
#define ASTL_VERSION 0.0.1
#endif

#define ASTL_VERSION_STRING "0.0.1"
#define ASTL_VERSION_MAJOR 0
#define ASTL_VERSION_MINOR 0
#define ASTL_VERSION_MICRO 1
```

For integrators, those macros describe the compile-time interface version: the API contract exposed by the headers that the client application was built against. By contrast, `astlVersionString()` and `astlVersion()` report the runtime implementation version of the ASTL library that is actually loaded.

In a well-matched installation, the compile-time header version and the runtime library version should agree. Consumers that need an explicit compatibility check can compare the `ASTL_VERSION_*` macros against the values returned by `astlVersion()` or compare `ASTL_VERSION_STRING` against `astlVersionString()`.

For ASTL 0.0.1:

- `astlVersionString()` returns `"0.0.1"`
- `astlVersion()` returns `{0, 0, 1}`

0.6 Core C API Conventions

0.6.1 Struct Versioning

Most ASTL parameter and properties structs begin with a `size` field. Callers must set:

```
params.size = sizeof(params);
```

ASTL provides convenience macros:

```
ASTL_INIT_STRUCT(astl_get_target_count_params_t, params,
                 .flags = 0,
                 .target_count = &target_count);
```

0.6.2 Caller-Allocated Buffers

Discovery and retrieval APIs generally follow a two-step pattern:

1. Query the count,
2. If the required count is `0`, skip the corresponding getter call,

3. Otherwise allocate a buffer (non-NULL, capacity `> 0`),
4. Set the `size` field of the first element when required,
5. Call the getter.

Helper macros:

```
ASTL_ALLOC_ARRAY(astl_target_props_t, targets, target_count);
ASTL_FREE_ARRAY(targets);
```

0.6.3 Opaque handles

The API uses opaque handles:

- `astl_target_handle_t`
- `astl_counter_handle_t`
- `astl_metric_handle_t`
- `astl_metric_group_handle_t`

Handles come from discovery APIs and are then reused for configuration, lifecycle, and retrieval.

0.6.4 Status Codes

Every C API returns `astl_status_code` defined in astl_errors.h.

Examples worth handling explicitly:

- `ASTL_STATUS_SUCCESS`
- `ASTL_STATUS_BAD_ARGUMENT`
- `ASTL_STATUS_INVALID_FLAG_VALUE`
- `ASTL_STATUS_NO_TARGET_FOUND`
- `ASTL_STATUS_NO_COUNTERS_FOUND`
- `ASTL_STATUS_NO_METRICS_FOUND`
- `ASTL_STATUS_NO_METRIC_GROUPS_FOUND`
- `ASTL_STATUS_INVALID_TARGET_HANDLE`
- `ASTL_STATUS_INVALID_COUNTER_HANDLE`
- `ASTL_STATUS_INVALID_METRIC_HANDLE`
- `ASTL_STATUS_INVALID_METRIC_GROUP_HANDLE`
- `ASTL_STATUS_COLLECTION_NOT_CONFIGURED`
- `ASTL_STATUS_COLLECTION_NOT_RUNNING`
- `ASTL_STATUS_COLLECTION_NOT_STOPPED`
- `ASTL_STATUS_COLLECTION_ALREADY_RUNNING`
- `ASTL_STATUS_COLLECTION_ALREADY_STOPPED`
- `ASTL_STATUS_COLLECTION_ALREADY_PAUSED`
- `ASTL_STATUS_FILE_ERROR`
- `ASTL_STATUS_BAD_CONFIGURATION`
- `ASTL_STATUS_INTERNAL_ERROR`

Use `astlStatusString()` to turn a status code into a readable message.

`astl_status_code` numeric assignments use explicit contiguous values (`0..44`) with `ASTL_STATUS_UNKNOWN_ERROR = -1`.

0.6.5 Ownership and Lifetimes

In properties structs, ASTL commonly returns `const char*` fields that point to immutable internal storage owned by ASTL. Callers must not free or modify them.

Collected samples use `astl_sample_t`, which intentionally has no `size` field.

0.7 Public Data Model

0.7.1 Platform

System-level properties are returned through `astl_platform_props_t`. These include:

- SoC name
- Vendor identifier
- OS name
- Kernel name, release, and version
- Firmware version
- Hostname
- Architecture

0.7.2 Target

A target is any ASTL-visible collection point. In practice, current targets usually represent:

- An SCMI telemetry source, or
- A libsensors chip.

Key target fields:

- `handle`
- `parent_handle`
- `name`
- `description`
- `id`

0.7.3 Counter

A counter is the lowest-level raw telemetry object ASTL exposes.

Key fields:

- `name`
- `description`
- `min_sampling_interval`
- `units`
- `formula`
- `value_type`
- `counter_type`

0.7.4 Metric

A metric is the processed, user-facing telemetry object ASTL expects most consumers to use.

Key fields:

- `name`
- `description`
- `units`
- `value_type`
- `metric_type`
- `identifier`

Metric kinds include:

- Value
- Finite-set value
- Event
- Delta
- Residency
- Rate

0.7.5 Metric Group

A metric group is a named, predefined grouping of metrics that can be discovered and configured together.

`astl_metric_group_props_t` is metadata-only:

- Handle
- Name
- Description

Current group API naming is:

- Target-scoped: `astlGetMetricGroupsOnTarget`
- Global: `astlGetMetricGroups`
- Global group membership count: `astlGetMetricGroupMetricCount`
- Global group membership: `astlGetMetricGroupMetrics`
- Target-filtered group membership count: `astlGetMetricGroupMetricCountOnTarget`
- Target-filtered group membership: `astlGetMetricGroupMetricsOnTarget`

The intended flow is:

1. Call the appropriate `...MetricCount` API
2. Allocate a metric-properties buffer of that size
3. Call the corresponding `...Metrics` API to fetch the metric properties

0.7.6 Sample

Collected counter and metric samples both use:

```
typedef struct _astl_sample_t {
  uint64_t     timestamp;
  astl_value_t value;
} astl_sample_t;
```

0.8 Shared Enums and Semantic Types

0.8.1 Units

`astl_units_t` covers telemetry units such as:

- None
- Ticks
- Seconds
- Celsius
- Joules
- Watts
- Volts
- Amps
- Bytes
- MB/s
- MHz
- RPM
- Count
- Percent

0.8.2 Value Types

`astl_value_type_t` identifies the active member of `astl_value_t`:

- `UINT8`
- `UINT16`
- `UINT32`
- `UINT64`
- `FLOAT32`
- `FLOAT64`
- `BOOL8`

0.8.3 Metric Identifiers

`astl_metric_identifier_t` provides semantic classification, including categories such as:

- Count
- Temperature
- Thermal limit
- Thermal throttle
- Energy
- Power
- Power limit
- Power throttle
- Frequency
- Voltage
- Current
- Bandwidth
- Fan speed
- Humidity

- Status

0.8.4 Collection Modes

`astl_collection_mode_t` supports:

- Sampling
- Immediate
- Snapshot

0.9 C API Inventory by Area

0.9.1 System Information

- `astlGetSystemInfo`: Return host or loaded-session platform properties.

0.9.2 Target Discovery

- `astlGetTargetCount`: Return the number of discoverable targets.
- `astlGetTargets`: Return target properties.

0.9.3 Counter Discovery

- `astlGetCounterCountOnTarget`: Return the number of counters on a target.
- `astlGetCountersOnTarget`: Return counter properties on a target.

0.9.4 Metric Discovery

- `astlGetMetricCountOnTarget`: Return the number of metrics on a target.
- `astlGetMetricsOnTarget`: Return metric properties on a target.

0.9.5 Metric State Discovery

- `astlGetMetricStateCountOnTarget`: Return the number of states for an enumerated metric.
- `astlGetMetricStatesOnTarget`: Return state metadata for an enumerated metric.

0.9.6 Metric-Group Discovery

- `astlGetMetricGroupCount`: Return the number of metric groups across all targets.
- `astlGetMetricGroupCountOnTarget`: Return the number of metric groups for a target.
- `astlGetMetricGroups`: Return global metric-group properties.
- `astlGetMetricGroupsOnTarget`: Return metric-group properties for a target.
- `astlGetMetricGroupMetricCount`: Return the number of metrics in a metric group across all targets.
- `astlGetMetricGroupMetrics`: Return all metrics that belong to a metric group, regardless of target.
- `astlGetMetricGroupMetricCountOnTarget`: Return the number of metrics in a metric group for a target.
- `astlGetMetricGroupMetricsOnTarget`: Return the metrics that belong to a target-specific group.

0.9.7 Collection Configuration

- `astlConfigureCounterCollectionOnTarget`: Configure raw counter collection for one target.
- `astlConfigureCounterCollection`: Configure raw counter collection across applicable targets.
- `astlConfigureMetricCollectionOnTarget`: Configure metric collection for one target.
- `astlConfigureMetricCollection`: Configure metric collection across applicable targets.
- `astlConfigureMetricGroupCollectionOnTarget`: Configure collection by metric group for one target.
- `astlConfigureMetricGroupCollection`: Configure collection by metric group across applicable targets.

All collection-configuration entry points use `astl_collection_params_t`, whose main fields are:

- `flags`
- `sampling_interval`
- `collection_mode`

0.9.8 Collection Lifecycle

- `astlReadImmediateOnTarget`: Trigger an immediate read on one target with configured counters or metrics.
- `astlReadImmediate`: Trigger immediate reads across targets with configured, started, or paused counters or metrics in the active scope.
- `astlStartCollectionOnTarget`: Start one configured target.
- `astlStartCollection`: Start collection across the active scope.
- `astlStartCollectionOnTargetPaused`: Start one target paused.
- `astlStartCollectionPaused`: Start collection paused across the active scope.
- `astlPauseCollectionOnTarget`: Pause one target.
- `astlPauseCollection`: Pause the active scope.
- `astlResumeCollectionOnTarget`: Resume one target.
- `astlResumeCollection`: Resume the active scope.
- `astlStopCollectionOnTarget`: Stop one target.
- `astlStopCollection`: Stop the active scope.

0.9.9 Session Persistence

- `astlSaveCollection`: Save in-memory collection state to a `.astl` archive.
- `astlLoadCollection`: Load a `.astl` archive.

0.9.10 Sample Retrieval

- `astlGetCounterSampleCountOnTarget`: Return the number of collected counter samples.
- `astlGetCounterSamplesOnTarget`: Return counter samples.
- `astlGetMetricSampleCountOnTarget`: Return the number of collected metric samples.
- `astlGetMetricSamplesOnTarget`: Return metric samples.

0.9.11 Statistics, Histogram, And States

- `astlGetMetricStatisticsOnTarget`: Return metric summary statistics.

- `astlGetMetricDiscreteHistogramBinCountOnTarget`: Return histogram bin count.
- `astlGetMetricDiscreteHistogramOnTarget`: Return discrete histogram bins.
- `astlGetMetricStatesOnTarget`: Return discrete states for finite-set metrics.

0.9.12 Cropping

- `astlCropSamplesOnTarget`: Crop all samples for one target.
- `astlCropSamples`: Crop samples across the active scope.

0.10 Typical Native Integration Flow

0.10.1 Flow 1: Enumerate The Platform

1. Call `astlGetTargetCount`
2. Allocate and call `astlGetTargets`
3. Choose a target
4. Call `astlGetCountersOnTarget`, `astlGetMetricsOnTarget`, or `astlGetMetricGroupsOnTarget`

0.10.2 Flow 2: Configure And Collect Metrics

1. Discover targets and metrics
2. Fill `astl_collection_params_t`
3. Call `astlConfigureMetricCollectionOnTarget`
4. Call `astlStartCollectionOnTarget`
5. Optionally pause and resume
6. Call `astlStopCollectionOnTarget`
7. Retrieve samples or statistics

0.10.3 Flow 2a: Configure And Read Immediately

1. Discover targets and counters, metrics, or metric groups
2. Fill `astl_collection_params_t`
3. Call one of the collection-configuration entry points
4. Call `astlReadImmediateOnTarget` or `astlReadImmediate`
5. Retrieve the captured samples

Immediate reads are available as soon as the target counters or metrics have been configured, and remain available while collection is started or paused.

0.10.4 Flow 3: Save And Reload

1. Complete or stop a collection
2. Call `astlSaveCollection`
3. Later call `astlLoadCollection`
4. Read metrics, samples, statistics, or histogram data from the loaded session

0.11 Example C Discovery Pattern

```
uint32_t target_count = 0;
ASTL_INIT_STRUCT(astl_get_target_count_params_t, count_params,
                 .flags = 0,
```

```
                .target_count = &target_count);

astl_status_code status = astlGetTargetCount(&count_params);
if (status != ASTL_STATUS_SUCCESS || target_count == 0) {
  return status;
}

ASTL_ALLOC_ARRAY(astl_target_props_t, targets, target_count);
ASTL_INIT_STRUCT(astl_get_targets_params_t, targets_params,
                .flags = 0,
                .targets = targets,
                .target_count = &target_count);

status = astlGetTargets(&targets_params);
if (status != ASTL_STATUS_SUCCESS) {
  ASTL_FREE_ARRAY(targets);
  return status;
}

/* inspect targets[0..target_count-1] */

ASTL_FREE_ARRAY(targets);
```

0.12 Python Wrapper Specification

0.12.1 Python Wrapper Positioning

The Python binding is currently the most ergonomic high-level wrapper in-tree. It is more than a raw FFI layer: it includes higher-level helpers for streaming, session management, diagnostics, optional DataFrame conversion, and derived metric calculation.

0.12.2 Public Modules

- `astl`: main public entry point.
- `astl._core`: Cython bridge to the C API.
- `astl.streaming`: polling and streaming helpers.
- `astl.session`: context-managed session API.
- `astl.diagnostics`: environment inspection.
- `astl.dataframe`: optional pandas conversion.
- `astl.derived`: deltas and rates.
- `astl.exceptions`: semantic exception mapping.

0.12.3 Current Public Capabilities

The current Python wrapper exposes:

- Version lookup
- Status-name lookup
- System info
- Target discovery

- Counter discovery
- Metric discovery
- Metric-group discovery
- Metric-group membership lookup
- Collection configuration
- Lifecycle control
- Immediate reads
- Save/load
- Sample retrieval
- Metric statistics
- Discrete histogram retrieval
- Metric-state retrieval
- Crop APIs
- Sync and async polling helpers
- Session context management

Representative public functions:

- `get_targets()`
- `get_counters(target)`
- `get_metrics(target)`
- `get_metric_groups()`
- `get_metric_groups_on_target(target)`
- `configure_metrics_on_target(...)`
- `start_collection(...)`
- `pause_collection(...)`
- `resume_collection(...)`
- `stop_collection(...)`
- `read_immediate(...)`
- `save_collection(...)`
- `load_collection(...)`
- `get_counter_samples(...)`
- `get_metric_samples(...)`
- `get_metric_statistics_on_target(...)`
- `get_metric_discrete_histogram_on_target(...)`
- `get_metric_states_on_target(...)`
- `poll_counter_once(...)`
- `stream_metric(...)`
- `Session`
- `diagnostics()`
- `to_dataframe(...)`
- `deltas(...)`
- `rates(...)`

0.12.4 Installation And Runtime Expectations

The Python package expects the native ASTL library to be installed or otherwise discoverable by the runtime loader.

In practice that means:

- ASTL native shared library present,
- Python package installed from `python/`,
- Loader path configured if the library is not in a standard location.

The wrapper reports its package version via `astl.__version__`, preferring installed package metadata and falling back to the repository `VERSION.md`.

0.13 Go Wrapper Specification

0.13.1 Go Wrapper Positioning

The Go wrapper is a supported in-tree wrapper around the public ASTL C API. It is a direct `cgo` wrapper rather than a large Go-native reimagining of the interface.

0.13.2 Current Exported Surface

The current package exports:

- Version helpers:
  - `VersionString()`
  - `VersionInfo()`
- System info:
  - `GetSystemInfo()`
  - `GetSystemInfoWithFlags(...)`
- Discovery:
  - `GetTargets()`
  - `GetCountersOnTarget(target)`
  - `GetMetricsOnTarget(target)`
  - `GetMetricGroups()`
  - `GetMetricGroupsOnTarget(target)`
  - `GetMetricGroupMetricCount(group)`
  - `GetMetricGroupMetrics(group)`
  - `GetMetricGroupMetricCountOnTarget(target, group)`
  - `GetMetricGroupMetricsOnTarget(target, group)`
- Configuration:
  - `ConfigureCountersOnTarget(...)`
  - `ConfigureCounters(...)`
  - `ConfigureMetricsOnTarget(...)`
  - `ConfigureMetrics(...)`
  - `ConfigureMetricGroupsOnTarget(...)`
  - `ConfigureMetricGroups(...)`
- Lifecycle:
  - `ReadImmediateOnTarget(...)`
  - `ReadImmediate()`
  - `StartCollectionOnTarget(...)`
  - `StartCollection()`
  - `StartCollectionOnTargetPaused(...)`
  - `StartCollectionPaused()`

  - `PauseCollectionOnTarget(...)`
  - `PauseCollection()`
  - `ResumeCollectionOnTarget(...)`
  - `ResumeCollection()`
  - `StopCollectionOnTarget(...)`
  - `StopCollection()`
- Persistence:
  - `SaveCollection(path)`
  - `LoadCollection(path, chunkSizeBytes)`
- Retrieval and analysis:
  - `GetCounterSamples(...)`
  - `GetMetricSamples(...)`
  - `GetMetricStatisticsOnTarget(...)`
  - `GetMetricDiscreteHistogramOnTarget(...)`
  - `GetMetricStatesOnTarget(...)`
- Crop operations:
  - `CropSamplesOnTarget(...)`
  - `CropSamples(...)`

0.13.3 Build And Link Requirements

The Go package expects:

- ASTL headers,
- A built ASTL shared library,
- `CGO_LDFLAGS` and runtime loader paths set correctly.

The in-repo workflow described by `Go/README.md` uses:

- `just build`
- `CGO_LDFLAGS` pointing at `build/debug/lib`
- `LD_LIBRARY_PATH` pointing at `build/debug/lib`

Current in-repo caveat:

- `Go/astl/astl.go` hardcodes include and library search paths under `build/debug`, so the checked-in Go wrapper currently assumes the debug preset rather than an arbitrary build preset.

0.14 Low-Level Telemetry Sources

Current ASTL discovery is centered on two backend families:

- `SCMI`: Status `primary`. Discovery source: Linux SCMI telemetry ioctl character devices, with fallback to the legacy SCMI telemetry sysfs interface. Notes: main implementation focus.
- `libsensors`: Status `supported`. Discovery source: lm-sensors / hwmon through libsensors. Notes: dynamic library dependency.

0.14.1 SCMI

ASTL currently focuses on the SCMI telemetry specification as surfaced through the Linux SCMI telemetry ioctl interface and the legacy Linux SCMI sysfs interface.

On current platforms, direct reads and periodic polling are the only supported SCMI sampling models. ASTL does not currently support event-driven or notification-based SCMI sampling.

0.14.1.1 What SCMI Requires On The System For live SCMI discovery to work, the following must exist:

1. At least one usable SCMI telemetry backend:
   - ioctl telemetry character devices under `/dev/scmi`, with device names such as `tlm_0`; override with `ASTL_SCMI_IOCTL_DEV_ROOT`.
   - Or a mounted legacy sysfs telemetry root under `/sys/fs/arm_telemetry`; override with `ASTL_SCMI_SYSFS_TELEMETRY_ROOT`.
2. A backend preference selected by `ASTL_SCMI_INTERFACE`, if the default is not desired.
   - `auto`: prefer ioctl when usable, otherwise fall back to sysfs.
   - `ioctl`: force the ioctl backend.
   - `sysfs`: force the legacy sysfs backend.
3. A telemetry implementation UUID that ASTL can normalize.
   - For ioctl, ASTL reads this through `SCMI_TLM_GET_INFO`.
   - For sysfs, ASTL reads `de_implementation_version`.
4. ASTL must be able to locate its configuration directory.
5. The config directory must contain:
   - one or more `scmi/public/**/repometa.json` fragments
   - The referenced SCMI specification JSON
   - one or more `metrics/**/platform_lookup.json` fragments
   - The referenced metrics declaration JSON

0.14.1.2 How ASTL Discovers SCMI Targets At runtime, ASTL:

1. Reads `ASTL_SCMI_INTERFACE`; unset, `auto`, or unknown values use automatic selection.
2. In automatic mode, probes ioctl devices first. If one or more usable ioctl targets are discovered, ASTL uses ioctl for SCMI discovery and collection.
3. If automatic ioctl discovery finds no usable targets, ASTL falls back to legacy sysfs discovery.
4. In forced `ioctl` mode, only ioctl devices are considered.
5. In forced `sysfs` mode, only legacy sysfs telemetry directories are considered.
6. Constructs targets named `scmi_<telemetry-subdirectory>`. ASTL keeps the stable target path in the legacy `tlm-N` form even when the kernel ioctl character device is named `tlm_N`.
7. Uses the normalized UUID to locate specification and metric metadata.

0.14.1.3 SCMI Metadata Required For Dynamic Metric Discovery SCMI target discovery alone is not enough for processed metric discovery. ASTL also needs shipped configuration metadata:

- `scmi/public/**/repometa.json`
  - Maps UUID to SCMI specification file, with paths relative to the fragment directory
- `metrics/**/platform_lookup.json`
  - Maps UUID to metrics declaration file, with paths relative to the fragment directory

The SCMI specification file tells ASTL what raw counters exist. The metrics declaration file tells ASTL how to present processed metrics on top of them.

0.14.1.4 Operational Note

In automatic mode, a missing ioctl device root is treated as "no ioctl targets" so ASTL can fall back to legacy sysfs. If the legacy sysfs root also does not exist, ASTL skips SCMI discovery rather than treating it as a fatal process-wide condition. Forced `ioctl` or forced `sysfs` mode disables the other backend.

0.14.2 Libsensors

ASTL also supports hwmon discovery through libsensors.

0.14.2.1 What Libsensors Requires On The System

For live libsensors discovery to work, the following must be true:

1. ASTL must have been built with libsensors support.
2. A compatible libsensors runtime library must be present and dynamically loadable.
   - Current warning text explicitly references the `libsensors5` runtime library
3. Required libsensors entry points must resolve successfully.
4. `sensors_init` must succeed using the system's default lm-sensors configuration.
5. The host must expose one or more detected chips through libsensors.
6. A discovered chip must expose at least one supported feature.

0.14.2.2 How ASTL Discovers Libsensors Targets

At runtime, ASTL:

1. Dynamically loads libsensors,
2. Initializes it,
3. Enumerates detected chips,
4. Scans chip features,
5. Creates one ASTL target per detected chip with at least one feature.

Current target naming:

- `libsensors_<chip-name>`

ASTL keeps the raw chip name in the target, even when the chip name includes a bus or PCI address.

0.14.2.3 Optional Exact And Family-Level Metric Metadata

For libsensors-backed metrics, ASTL optionally looks for a metrics declaration file under:

`<ASTL_CONFIG_DIR>/metrics/libsensors/`

or the equivalent built-in config path under the packaged ASTL config directory.

Lookup order for a discovered target is:

1. Exact target file: `libsensors_<chip-name>.json`
2. Fallback family file(s) built by progressively trimming the trailing `-...` suffixes from the chip name

Examples:

```
libsensors_nvme-pci-40100.json   # exact target file
libsensors_nvme-pci.json         # family fallback file
libsensors_bnxt_en-pci.json      # family fallback file
```

Behavior:

- If no file exists, ASTL still registers discovered readable metrics using built-in defaults,
- An exact target file acts as an allowlist and metadata source,
- A fallback family file adds metadata without hiding undeclared discovered metrics,
- Declaration files can attach descriptions, groups, identifiers, formulas, and shared inheritance via `"extends"`,
- Derived libsensors metrics can also be expanded from `"derived_metrics"` declarations.

0.14.2.4 Stable Libsensors Metric Naming When a family-level libsensors declaration file is used, ASTL now derives stable metric-name prefixes from the chip family plus a runtime instance index rather than from the raw address-bearing chip name.

Examples:

```
nvme-pci-1_Composite
nvme-pci-2_Composite
bnxt_en-pci-1_temp1
bnxt_en-pci-2_temp1
```

That means:

- Raw target names may still include the discovered chip name and PCI address,
- Metric names do not need to embed that address when ASTL can resolve a matching family-level declaration file,
- Higher-level tooling can build stable presentation names on top of those family-instance metric names.

0.15 ASTL Runtime Configuration Resolution

Live discovery uses one selected explicit, adjacent, or bundled configuration source.

The procfs collector exposes the `sysutil` target with `cpu.total.utilization`, `cpu.N.utilization`, `memory.total`, `memory.used`, and `memory.utilization`. These replace the old procfs target and utilization names without aliases. Collector selection remains `ASTL_COLLECTORS=procfs`. See System utilization for migration and sampling details.

Backend paths are resolved independently from the configuration directory:

- `ASTL_COLLECTORS`: comma-separated, case-insensitive allowlist for live collector discovery. Supported values are `scmi`, `libsensors`, and `procfs`. Unset or empty enables all collectors compiled into ASTL. Unknown names, or names unavailable in the current build, cause initialization to return `ASTL_STATUS_BAD_CONFIGURATION`. Saved `.astl` sessions retain their recorded topology and are not filtered by this setting.
- `ASTL_SCMI_INTERFACE`: `auto`, `ioctl`, or `sysfs`. Defaults to `auto`.
- `ASTL_SCMI_IOCTL_DEV_ROOT`: ioctl telemetry character-device root. Defaults to `/dev/scmi`.
- `ASTL_SCMI_SYSFS_TELEMETRY_ROOT`: legacy sysfs telemetry root. Defaults to `/sys/fs/arm_telemetry`.
- `ASTL_PROCFS_ROOT`: procfs root used for topology discovery, metric discovery, and collection. Defaults to `/proc`.

ASTL uses a single configuration source, selected in this order:

1. The directory specified by `ASTL_CONFIG_DIR`, when set.

2. A `config/` directory beside the loaded ASTL library, or beside the executable when ASTL is statically linked.
3. The configuration embedded in the library.

Sources are not combined. No OS-specific user or system config directories are searched. An invalid explicit directory (including an empty `ASTL_CONFIG_DIR`) or invalid contents in the selected directory cause an error, without falling back to another source.

Configuration references (`extends`, `metrics_file`, and `specification_file`) must be relative to the referring JSON file and resolve to a JSON file within the selected config directory. Absolute paths and paths escaping that directory are rejected.

Every build embeds configuration. `ASTL_EMBED_CONFIG_DIR` selects the prepared input tree at configure time (default: this project's `config/`). Apply ossmosis and release-profile filtering before configuration. `ASTL_RELEASE_PROFILE` identifies the embedded input set in the generated resource manifest.

The resource tree contains `metrics/`, `groups/`, and `scmi/public/`.

0.16 What Dynamic Discovery Means in Practice

0.16.1 Live Host Discovery

Live host discovery is possible when:

- The low-level source is present,
- ASTL can reach the source,
- ASTL can read the source metadata needed to identify it,
- ASTL can load matching configuration resources.

0.16.2 Loaded-Session Mode

If a caller uses `astlLoadCollection`, ASTL can expose the loaded session's system information and collected data without requiring the live host telemetry interfaces to be present.

That is useful for:

- Offline analysis,
- Replay,
- Post-processing on a different machine,
- Environments without direct SCMI or libsensors access.

0.17 Supported Platforms

ASTL itself is backend-driven rather than hard-coding a closed platform list, but the current tree ships explicit libsensors metric declarations for targets observed on two concrete platform selectors:

- `Arm AGI CPU` Selector: `vendor_id = "Arm"`, `soc_name = "AGI CPU"`. Scope: SCMI per-core telemetry plus NVMe and Broadcom `bnxt_en` NIC thermal telemetry. Backing target families: `libsensors_nvme-pci-*`, `libsensors_bnxt_en-pci-*`.
- `Ampere Altra` Selector: `vendor_id = "System76"`, `soc_name = "jep106:0a16"`. Scope: APM X-Gene platform sensor, NVMe, Broadcom `bnxt_en` NICs, and System76

Thelio I/O fan telemetry. Backing target families: `libsensors_apm_xgene-isa-*`, `libsensors_nvme-pci-*`, `libsensors_bnxt_en-pci-*`, `libsensors_system76_thelio_io-hid-3-5`.

The lists below show the current raw ASTL metric names that back those profiles.

0.17.1 Arm AGI CPU

Raw ASTL metrics:

- `TEMP_PRESENT_CORE_<n>`: SCMI per-core temperature metrics expanded into one raw ASTL metric per discovered core instance. Units: `celsius`.
- `bnxt_en-pci-1_temp1`: NIC temperature reported by the `bnxt_en` PCI sensor. Units: `celsius`.
- `bnxt_en-pci-1_temp1_thermal_limit_critical`: Configured critical temperature threshold for the `bnxt_en` PCI sensor. Units: `celsius`.
- `bnxt_en-pci-1_temp1_thermal_limit_emergency`: Configured emergency temperature threshold for the `bnxt_en` PCI sensor. Units: `celsius`.
- `bnxt_en-pci-1_temp1_thermal_limit_high`: Configured high temperature threshold for the `bnxt_en` PCI sensor. Units: `celsius`.
- `bnxt_en-pci-2_temp1`: NIC temperature reported by the `bnxt_en` PCI sensor. Units: `celsius`.
- `bnxt_en-pci-2_temp1_thermal_limit_critical`: Configured critical temperature threshold for the `bnxt_en` PCI sensor. Units: `celsius`.
- `bnxt_en-pci-2_temp1_thermal_limit_emergency`: Configured emergency temperature threshold for the `bnxt_en` PCI sensor. Units: `celsius`.
- `bnxt_en-pci-2_temp1_thermal_limit_high`: Configured high temperature threshold for the `bnxt_en` PCI sensor. Units: `celsius`.
- `nvme-pci-1_Composite`: Composite NVMe drive temperature reported by the controller sensor. Units: `celsius`.
- `nvme-pci-1_Composite_alarm`: Composite thermal alarm status reported by the NVMe drive. Units: `none`.
- `nvme-pci-1_Composite_thermal_limit_critical`: Configured critical composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-1_Composite_thermal_limit_high`: Configured high composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-1_Composite_thermal_limit_low`: Configured low composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-1_Sensor_1`: Additional NVMe drive temperature reported by sensor
  1. Units: `celsius`.
- `nvme-pci-1_Sensor_2`: Additional NVMe drive temperature reported by sensor 2. Units: `celsius`.
- `nvme-pci-1_Sensor_3`: Additional NVMe drive temperature reported by sensor 3. Units: `celsius`.
- `nvme-pci-2_Composite`: Composite NVMe drive temperature reported by the controller sensor. Units: `celsius`.
- `nvme-pci-2_Composite_alarm`: Composite thermal alarm status reported by the NVMe drive. Units: `none`.
- `nvme-pci-2_Composite_thermal_limit_critical`: Configured critical composite temperature threshold for the NVMe drive. Units: `celsius`.

- `nvme-pci-2_Composite_thermal_limit_high`: Configured high composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-2_Composite_thermal_limit_low`: Configured low composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-2_Sensor_1`: Additional NVMe drive temperature reported by sensor
  1. Units: `celsius`.
- `nvme-pci-2_Sensor_2`: Additional NVMe drive temperature reported by sensor 2. Units: `celsius`.
- `nvme-pci-2_Sensor_3`: Additional NVMe drive temperature reported by sensor 3. Units: `celsius`.

0.17.2 Ampere Altra

Raw ASTL metrics:

- `system76_thelio_io-hid-3-5_Aux_Fan`: Auxiliary chassis fan speed reported by the System76 Thelio I/O controller. Units: `RPM`.
- `system76_thelio_io-hid-3-5_CPU_Fan`: CPU cooling fan speed reported by the System76 Thelio I/O controller. Units: `RPM`.
- `apm_xgene-isa-0000_CPU_power`: CPU power draw reported by the APM X-Gene platform sensor. Units: `watts`.
- `system76_thelio_io-hid-3-5_GPU_Fan`: GPU cooling fan speed reported by the System76 Thelio I/O controller. Units: `RPM`.
- `system76_thelio_io-hid-3-5_Intake_Fan`: Chassis intake fan speed reported by the System76 Thelio I/O controller. Units: `RPM`.
- `apm_xgene-isa-0000_IO_power`: I/O power draw reported by the APM X-Gene platform sensor. Units: `watts`.
- `bnxt_en-pci-1_temp1`: NIC temperature reported by the `bnxt_en` PCI sensor. Units: `celsius`.
- `bnxt_en-pci-2_temp1`: NIC temperature reported by the `bnxt_en` PCI sensor. Units: `celsius`.
- `nvme-pci-1_Composite`: Composite NVMe drive temperature reported by the controller sensor. Units: `celsius`.
- `nvme-pci-1_Composite_alarm`: Composite thermal alarm status reported by the NVMe drive. Units: `none`.
- `nvme-pci-1_Composite_thermal_limit_critical`: Configured critical composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-1_Composite_thermal_limit_high`: Configured high composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-1_Composite_thermal_limit_low`: Configured low composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-1_Sensor_2`: Additional NVMe drive temperature reported by sensor 2. Units: `celsius`.
- `nvme-pci-2_Composite`: Composite NVMe drive temperature reported by the controller sensor. Units: `celsius`.
- `nvme-pci-2_Composite_alarm`: Composite thermal alarm status reported by the NVMe drive. Units: `none`.
- `nvme-pci-2_Composite_thermal_limit_critical`: Configured critical composite tempera-

ture threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-2_Composite_thermal_limit_high`: Configured high composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-2_Composite_thermal_limit_low`: Configured low composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-2_Sensor_2`: Additional NVMe drive temperature reported by sensor 2. Units: `celsius`.
- `nvme-pci-3_Composite`: Composite NVMe drive temperature reported by the controller sensor. Units: `celsius`.
- `nvme-pci-3_Composite_thermal_limit_critical`: Configured critical composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-3_Composite_thermal_limit_high`: Configured high composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-3_Composite_thermal_limit_low`: Configured low composite temperature threshold for the NVMe drive. Units: `celsius`.
- `nvme-pci-3_Sensor_2`: Additional NVMe drive temperature reported by sensor 2. Units: `celsius`.
- `apm_xgene-isa-0000_SoC_Temperature`: SoC temperature reported by the APM X-Gene platform sensor. Units: `celsius`.

0.18 Release Readiness Review

0.18.1 Blind Spots And Gaps To Address Before First Release

- `astlConfigureCounterCollection()` still returns `ASTL_STATUS_NOT_IMPLEMENTED` for cross-target counter configuration. Only the on-target counter configuration path is implemented today.
- Pause and resume are implemented by `astlPauseCollectionOnTarget()`, `astlPauseCollection()`, `astlResumeCollectionOnTarget()`, and `astlResumeCollection()`. The calls propagate the orchestrator result through the public status-code mapping. Collector-dependent unsupported operations return `ASTL_STATUS_PAUSE_UNSUPPORTED` or `ASTL_STATUS_RESUME_UNSUPPORTED`; the target transitions to PAUSED or STARTED only after the collector succeeds.
- Metric-manager protobuf serialization does not yet cover all metric types and still omits counters. Residency metric deserialization is still treated as not implemented, and counters are explicitly cleared on deserialize.
- SCMI residency metrics with non-zero base10 unit modifiers are rejected as not implemented to avoid silently producing wrong values.
- Some SCMI data-event directory paths still log as not implemented and are skipped rather than fully supported.
- If `config/groups/metric_groups.json` is missing, ASTL only logs a warning. Metrics that declare group membership can then fail later during registration instead of failing immediately at startup.
- The Go wrapper still hard-codes `build/debug` include and library paths in its `cgo` directives, so it is not yet preset-agnostic.
- Python packaging still mixes a top-level `LICENSE` source with `python/pyproject.toml` `license-files = ["LICENSE"]`, which is why local packaging/test flows can recreate an untracked `python/LICENSE` artifact.

0.18.2 Pre-Release Compatibility Code Currently Present

- The C API already carries explicit struct-size version negotiation and `OLD_*/NEW_*` status codes even though no external ABI has shipped yet.
- `ASTL_STATUS_DEPRECATED_API` is already exported through the C API and both in-tree wrappers even though there is no released deprecated surface to route through it.
- `python/astl/__init__.py` documents an additive backward-compatibility policy before ASTL has had a first public release.

0.19 Release Notes

- ASTL 0.0.1 currently supports dynamic discovery from SCMI telemetry ioctl, legacy SCMI sysfs, and libsensors-backed targets, plus offline replay through `.astl` session files.
- For address-bearing libsensors devices such as NVMe and `bnxt_en` NICs, raw target names may still include the discovered chip name or PCI address, while metric names use stable family-instance tokens when family-level declarations are available.
- Session replay preserves metric and metric-group data, but counter data is not yet part of the serialized metric-manager payload.
- Pause and resume are implemented for individual targets and through the all-target APIs. The all-target calls attempt the operation on every target and return the first non-success status, or `ASTL_STATUS_SUCCESS` if all calls succeed. Support depends on the selected collector and valid lifecycle state.
- The Python wrapper is the most feature-complete high-level interface in-tree. The Go wrapper is now part of the supported wrapper surface, but it remains build-layout-sensitive.

0.20 Recommended Consumer Guidance

0.20.1 If You Want The Most Stable Integration Surface

Use the C API.

Reasons:

- It is the canonical public interface,
- Both wrappers map back to it,
- The exported headers are the clearest contract boundary.

0.20.2 If You Want The Most Ergonomic Scripting And Analysis Layer

Use the Python wrapper.

Reasons:

- Broader high-level convenience surface,
- Streaming helpers,
- Session helpers,
- Diagnostics,
- Optional dataframe conversion.

0.20.3 If You Want Native Go And Want A Wrapper Close To The C API

Use the Go wrapper.

Reasons:

- It maps closely to the C API,
- It is now part of the supported in-tree wrapper surface,
- It currently has a smaller and more direct scope than the Python layer.

0.21 Notes and Caveats

- This document is intentionally implementation-aware where that helps integrators, especially for discovery and configuration requirements.
- The public headers remain the final source of truth for exact field definitions, enum values, and ABI details.
- The Go wrapper is supported, but currently assumes repo-local `build/debug` headers and libraries unless the caller overrides the `cgo` build settings.
- The Python wrapper is richer than the Go wrapper and currently acts as the most feature-complete high-level wrapper in-tree.
- Metric-group naming in older drafts may refer to `astlGetMetricGroups`; the current API uses `astlGetMetricGroupsOnTarget`.

0.22 Reference Pointers

For deeper detail, start with:

- `include/astl/astl.h`
- `include/astl/astl_telemetry.h`
- `include/astl/astl_errors.h`
- `include/astl/astl_version.h.in`
- `README.md`
- `python/doc/README.md`
- `python/doc/USER_GUIDE.md`
- `Go/README.md`